MEDS Lab - Module 4

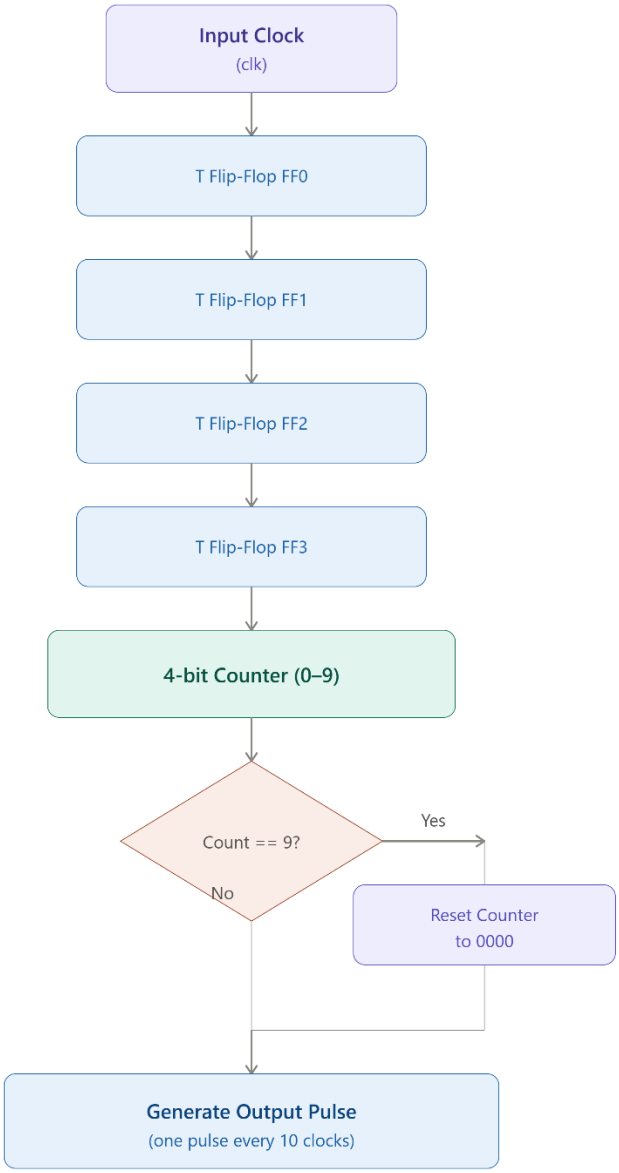
Section 10 Report: Counters & Timing Applications

Name: Muhammad Kaleem Faryad
University: UET Lahore

Task 12: Modulo-10 Frequency Divider Using T Flip-Flops

1. **EDA Playground Link:** https://edaplayground.com/x/AAE9

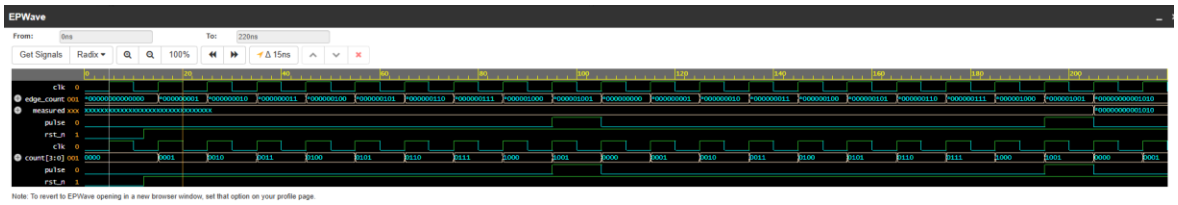
2. **Block Diagram:**



3. **Simulation Output Screenshot:**

```
Log   Share
CPU time: .673 seconds to compile + .774 seconds to elab + .464 seconds to link
Chronologic VCS simulator copyright 1991-2025
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full64; Runtime version X-2025.06-SP1_Full64;  Jul 22 14:36 2026
[PASS] Divider Ratio = 10
$finish called from file "testbench.sv", line 65.
$finish at simulation time                 225
           V C S   S i m u l a t i o n   R e p o r t
Time: 225 ns
CPU Time:      0.520 seconds;       Data structure size:   0.0Mb
Wed Jul 22 14:36:59 2026
Finding VCD file...
./dump.vcd
[2026-07-22 18:36:59 UTC] Opening EPWave...
Done
```

4. **Waveform Screenshot:**



5. **Explanation:** The objective of this task was to design a **Modulo-10 Frequency Divider** using **T flip-flops**. A frequency divider reduces the frequency of an input clock by a fixed factor. Since the required division factor is 10, the counter repeatedly counts from **0 to 9** and then returns to **0**. One output pulse is generated whenever the counter reaches its terminal count, producing one pulse for every ten input clock cycles.

   The design uses **four T flip-flops**, which together form a **4-bit synchronous counter**. Although only ten states (0–9) are required, four flip-flops are sufficient because a 4-bit counter can represent values from **0 to 15**. Additional logic detects when the counter reaches **9 (1001_2)**. On the next clock edge, the counter is reset to **0000**, preventing it from counting beyond 9.

   A terminal-count detection circuit continuously compares the current counter value with decimal **9**. Whenever this value is detected, the output **pulse** is asserted for one clock cycle, and the counter is simultaneously reset to zero. As a result, the output pulse repeats once every ten input clock cycles, achieving the required **divide-by-10** operation.

   The testbench verifies the design using a **self-checking approach**. It first applies an active-low reset to initialize the counter. After reset is released, the testbench waits for the first output pulse and then counts the number of rising clock edges until the next pulse occurs. If the measured count is exactly **10**, the testbench displays a **PASS** message; otherwise, it displays a **FAIL** message. This automatically confirms that the divider produces one output pulse after every ten input clock cycles without requiring manual inspection of the waveform.

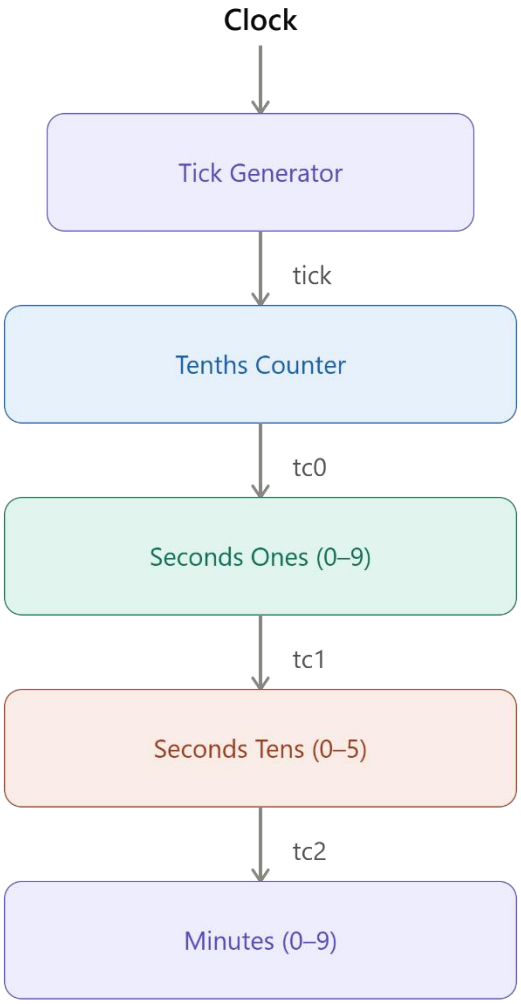
   The simulation results confirmed that the pulse was generated after every ten clock

cycles, demonstrating that the modulo-10 counter correctly functions as a **divide-by-10 frequency divider**.

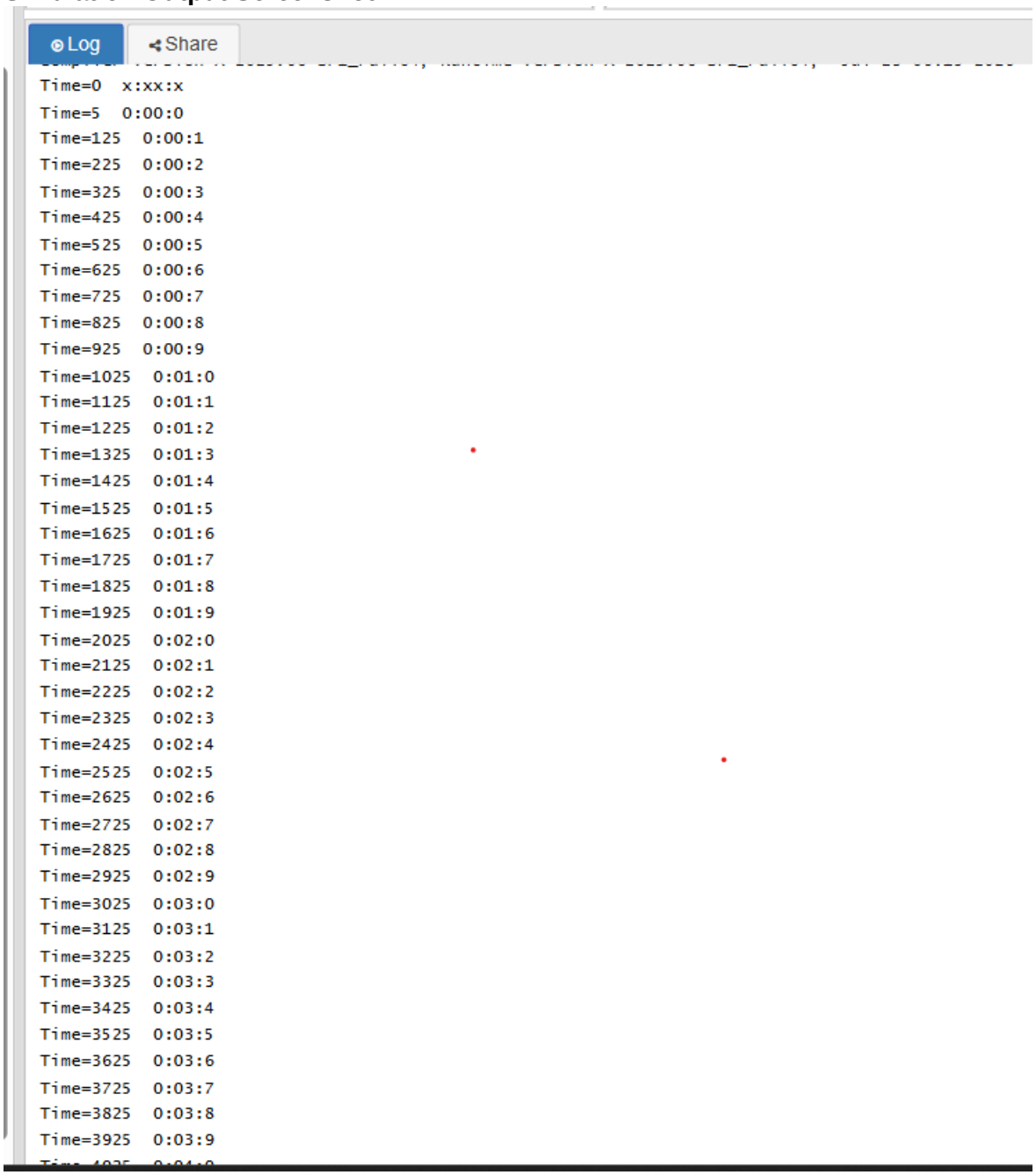
Task 13: 4-Digit Stopwatch

1. **EDA Playground Link:** https://edaplayground.com/x/wiPc

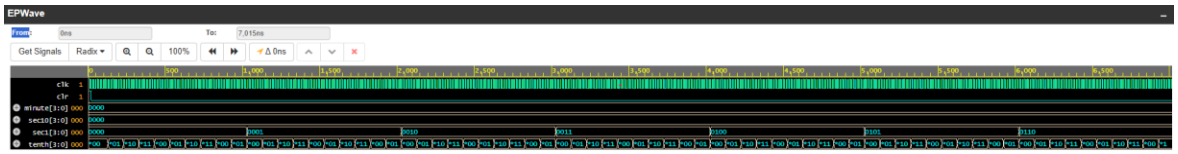
2. **Block Diagram:**



3. Simulation Output Screenshot:



4. Waveform Screenshot:



5. Calculations:

Frequency: The stopwatch increments its rightmost digit once every **0.1 second**.
Therefore, a tick signal with a frequency of **10 Hz** must be generated from the input clock.

Assume the simulation clock has a period of **10 ns**, which corresponds to an input clock frequency of

Fin = 1/Tclk = 1/10×10^-9 = 100MHz

The required stopwatch tick frequency is:

Ftick = 1/ 0.1=10 Hz

Using the frequency divider equation,

N= fin / ftick

Substituting the values,

N=100,000,000/10=10,000,000

Therefore, the tick generator must produce one output pulse after every **10,000,000 input clock cycles**.

Because simulating ten million clock cycles for every stopwatch tick would require a very long simulation time, a smaller divider value (such as 10) was used in the simulation. Although the divider is reduced for faster simulation, the operating principle remains exactly the same as the real hardware implementation.

Duty Cycle Calculation: The tick generator produces a pulse that remains HIGH for only **one clock cycle** every time the counter reaches its terminal count.

The duty cycle is calculated as:

Duty Cycle = Pulse Width×100% / Output Period

Since the pulse width is one clock period,

Duty Cycle = 1/N×100%

Real Hardware
For the required divider,

N=10,000,000

Therefore,

Duty Cycle = 1/10,000,000×100

Duty Cycle=0.00001%

Thus, the output pulse remains HIGH for only a very small portion of each output period.

Simulation

For simulation, the divider was chosen as

N = 10

Hence,

Duty Cycle = 1/10×100 = 10%

This larger duty cycle makes the output pulse easier to observe during simulation while preserving the same counter operation.

Pulse Width Calculation

The pulse generated by the tick generator remains HIGH for exactly one input clock period.

The clock period is:

$$T_{clk} = 10ns$$

Therefore,

$$\text{Pulse Width} = T_{clk} = 10ns$$

Both the real hardware implementation and the simulation generate a pulse that is one clock period wide. The only difference is the divider value used to determine how often the pulse occurs.

6. **Explanation:** A tick generator is used to convert the high-frequency input clock into a much slower timing pulse required by the stopwatch. The stopwatch should advance its tenths digit once every **0.1 second**, corresponding to a frequency of **10 Hz**. Since the simulation clock operates at **100 MHz**, the clock must be divided by **10,000,000** to obtain the required tick frequency. This is achieved by using a counter that counts the input clock cycles and generates a pulse whenever the terminal count is reached.
   The generated tick pulse remains HIGH for only one clock cycle and then returns LOW. This narrow pulse acts as the enable signal for the tenths counter. Every tick increments the tenths digit by one. When the tenths counter completes a full cycle (9 to 0), it enables the seconds counter. Similarly, the seconds counters enable the minutes counter through their terminal count signals. This cascading mechanism allows the stopwatch to count correctly from **0:00:0** to **9:59:9**.
   In the simulation, a smaller divider was used instead of 10,000,000 to reduce the execution time. Although the divider value is smaller, the functional behavior of the stopwatch remains identical because the design logic is unchanged.